

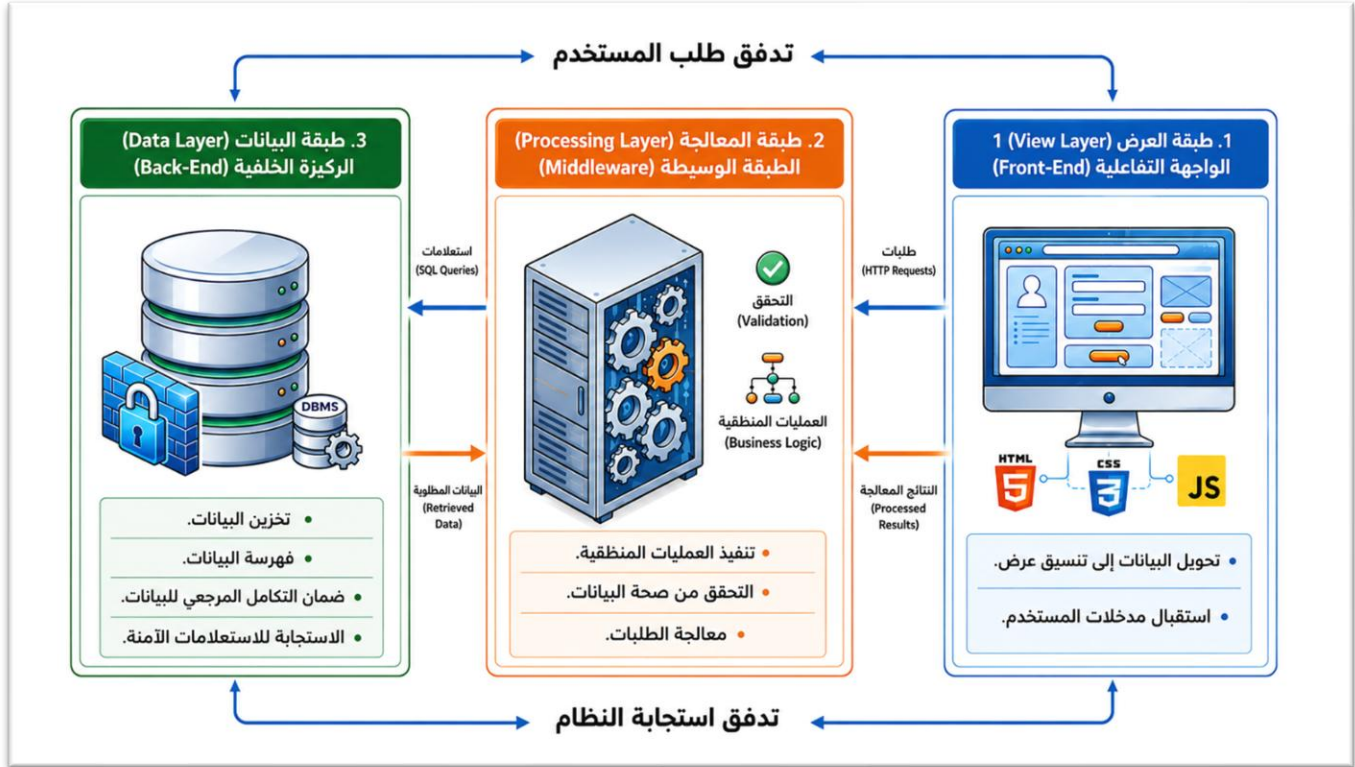
## برمجة الويب

### المفهوم:

تُعرّف برمجة الويب بأنها: عملية إنشاء مواقع، وتطبيقات ويب ديناميكية وتفاعلية تتكون من جزأين: الواجهة الأمامية (Frontend) التي يتعامل معها المستخدم مباشرة، والواجهة الخلفية (Backend) التي تتعامل مع منطق التطبيق والخادم وقواعد البيانات. تلعب قواعد البيانات دورًا حيويًا في الواجهة الخلفية، حيث توفر آلية تخزين واسترجاع البيانات اللازمة لتشغيل التطبيقات الديناميكية، مثل: مواقع التجارة الإلكترونية، ومنصات التواصل الاجتماعي، وأنظمة إدارة المحتوى. (Novaliendry et al., 2021)

في سياق تطبيقات الويب الديناميكية، تقوم الواجهة الخلفية (Back-End) بالدور المحوري في إدارة العمليات التوليدية والمنطقية للنظام. ويشير (Muittari, 2020) إلى أن هذه الطبقة مسؤولة عن التوليد الفوري للمحتوى (Real-time Content Generation) استجابةً للاحتياجات الآنية؛ مما يتيح تقديم تجربة مستخدم مخصصة (Personalized) تتكيف مع تفضيلات المتعلم وبياناته المخزنة مسبقًا. وتقنيًا، تتم هذه العملية عبر دمج البيانات المسترجعة من قاعدة البيانات داخل أماكن مخصصة في قوالب HTML الهيكلية، لإنتاج صفحات ويب متكاملة، بالتوازي مع تنفيذ مهام تشغيلية خلفية، كإرسال الإشعارات والتنبيهات، أثناء دورة الاستجابة.

شكل (13)



وتُعد الواجهة الخلفية بمثابة البنية التحتية البرمجية المسؤولة عن المنطق التشغيلي للنظام؛ حيث أوضحت (Malikova & Kaarov, 2022) أن هذه الطبقة تعمل وفق مبدأ الفصل المنطقي وتعتمد على لغات برمجية متخصصة (مثل: PHP, Java, Node.js) لتوفير واجهات برمجة تطبيقات (APIs) وتعمل هذه الواجهات كطبقة تُمكن الواجهة الأمامية من استدعاء الخدمات والتفاعل مع البيانات بسلاسة، دون الحاجة للخوض في تعقيدات التنفيذ الداخلي للكود؛ مما يعزز من كفاءة النظام وقابليته للتطوير.

وتُعد قواعد البيانات والجانب الخلفي (Back-End) من المكونات الأساسية لأي تطبيق ويب حديث. يتكامل هذان الجزءان لإنشاء تجربة مستخدم ديناميكية وفعالة، حيث يعمل الجانب الخلفي كوسيط حيوي بين واجهة المستخدم (Front-End) وقواعد البيانات التي تخزن جميع المعلومات

الضرورية. هذا التفاعل يضمن معالجة سلسلة للبيانات واستجابة سريعة لطلبات المستخدمين (Bogutskii, 2025).

### هيكلية قواعد بيانات الويب (Web-Based Database Architecture) :

يعتمد بناء نظام الويب على **بنية** ثلاثية الطبقات (Three-Tier Architecture)، وهو النموذج المعياري الأكثر كفاءة لفصل العمليات وتأمين البيانات. تتكامل الطبقات الثلاث لضمان تدفق البيانات بسلاسة من المستخدم إلى **المخزن الرئيس** وبالعكس، ويمكن توصيفها في 3 طبقات كما وضحتها (Rodríguez-Campos et al., 2024). **فيما يلي شرح للطبقات الثلاث:**

#### 1. طبقة العرض (View Layer):

تمثل هذه الطبقة الواجهة التفاعلية (Front-End) التي تظهر للمستخدم عبر متصفح الويب، وتُبنى باستخدام تقنيات العميل القياسية (HTML, CSS, JavaScript). تكمن وظيفتها المحورية في تحويل البيانات المجردة إلى تنسيق بصري مفهوم، واستقبال مدخلات المستخدم عبر النماذج التفاعلية، ومن ثم إرسال هذه الطلبات (HTTP Requests) إلى خادم الويب للمعالجة، دون أن تحتوي هذه الطبقة على أي منطق برمجي معقد أو اتصال مباشر بقاعدة البيانات.

#### 2. طبقة المعالجة (Processing Layer):

تُعرف بالطبقة الوسيطة (Middleware)، وهي العقل المدبر للنظام الذي يستضيف الكود البرمجي باستخدام لغات البرمجة مثل لغة PHP، تقوم هذه الطبقة باستلام الطلبات من طبقة العرض، وتنفيذ العمليات المنطقية (Business Logic) مثل التحقق من صحة البيانات (Validation) والحسابات، **وتؤدي** دور الوسيط الآمن الذي يقوم بإنشاء الاتصال مع قاعدة البيانات لتنفيذ عمليات الاستعلام، ومن ثم تجهيز النتائج لإعادة إرسالها إلى المستخدم.

#### 3. طبقة البيانات (Data Layer):

تُشكل هذه الطبقة الركيزة الخلفية (Back-End) للنظام، وتتضمن المكونات الفعلية لقاعدة البيانات (Database) المسؤولة عن تخزين المعلومات. تخضع هذه الطبقة لإدارة كاملة من قبل نظام إدارة قواعد البيانات مثل MySQL، حيث تتولى مهام حفظ البيانات، وفهرستها، وضمان تكاملها المرجعي. وعلاوة

على ذلك، تعمل هذه الطبقة كمستودع بيانات معزول، لا يستجيب إلا للاستعلامات الهيكلية (SQL Queries) الصادرة حصراً عن طبقة التطبيق؛ مما يحقق مستوى عالٍ من الأمان وحماية البيانات من الوصول المباشر غير المصرح به.

- **دورة حياة تطوير قاعدة بيانات الويب<sup>1</sup> (Web Database Development Life Cycle):**

تتألف دورة حياة تطوير قاعدة بيانات الويب من أربعة مراحل، بيانها كالتالي:

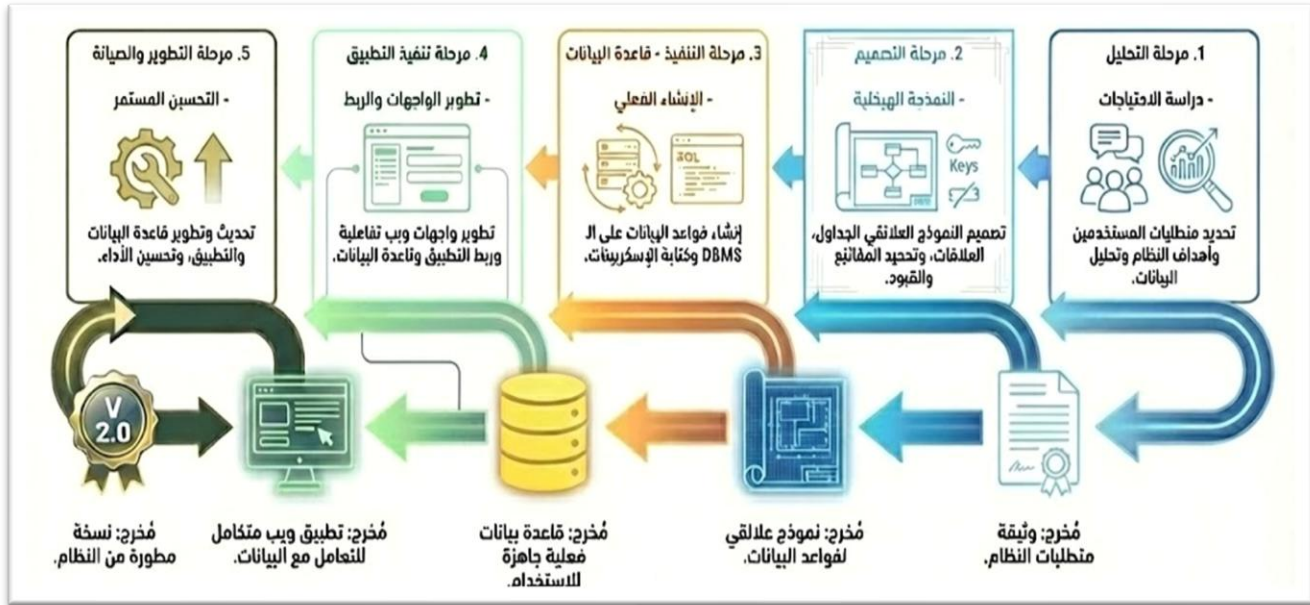
1. **مرحلة التحليل:** وتشمل تحديد متطلبات المستخدمين، أهداف النظام ودراسة الاحتياجات، وتحليل البيانات المطلوبة، وتمثل مخرجات هذه المرحلة وثيقة متطلبات النظام.
2. **مرحلة التصميم:** وتشمل تصميم النموذج العلائقي للجدول والعلاقات، وتحديد المفاتيح، والقيود ومخرجات هذه المرحلة نموذج علائقي لقاعدة البيانات ERD.
3. **مرحلة التنفيذ (قاعدة البيانات):** ويتم فيها إنشاء قاعدة البيانات من خلال نظام إدارة قواعد البيانات، ومخرجات هذه المرحلة قاعدة بيانات فعلية جاهزة للاستخدام.
- في هذه المرحلة أيضا يتم بناء الواجهة:** وتشمل تطوير واجهات ويب تفاعلية، وربط التطبيق بقاعدة البيانات، ومخرجات هذه المرحلة نموذج ويب للتعامل مع البيانات.
4. **مرحلة التطوير والصيانة:** وتشمل تحديث وتطوير قاعدة البيانات والتطبيق، وتحسين الأداء، ومخرجات هذه المرحلة نسخة مطورة من نظام قاعدة البيانات والتطبيق.

الفيديوهات على قناة اليوتيوب من محتوى منحة وزارة الاتصالات والمعلومات

شكل (14)

دورة حياة تطوير قاعدة بيانات الويب

<sup>1</sup> <https://youtu.be/KddAhvKRHa0?si=AWd9qx6I1kfQqitU>



#### • مهارات برمجة قواعد بيانات الويب ( الجانب الخلفي للويب):

يرتكز بناء المعمارية البرمجية للواجهة الخلفية على تكامل مجموعة من اللغات وأطر العمل المتطورة، التي تشكل الأساس الذي تقوم عليه معالجة البيانات والعمليات المنطقية في تطبيقات الويب الحديثة حيث لا تعمل قواعد البيانات في بيئة الويب بمعزل عن التطبيق، بل تمثل جزءاً من منظومة تقنية تعتمد على معمارية (العميل/الخادم). ولتحقيق التفاعل الديناميكي في هذه المنظومة، يستلزم هذا الأمر توظيف أدوات برمجية محددة لسد الفجوة بين واجهة المستخدم والمخزن الرقمي للبيانات. ينقسم هذا التوظيف البرمجي إلى مستويين: الأول يعنى بلغة البيانات (SQL) المسؤولة عن سلامة وبناء المحتوى، والثاني يعنى بلغة المنطق والتطبيق (Back-End) المسؤولة عن استقبال طلبات المستخدم وتوجيهها، لذا يتناول هذا الجزء المهارات التقنية الأساسية التي يحتاجها الطلاب **لبرمجة** قواعد بيانات الويب.

#### - لغات البرمجة وأطر العمل (Programming Languages and Frameworks): عادةً

ما يتم تنفيذ عمليات التطوير من جانب الخادم باستخدام لغات برمجة مثل JavaScript في بيئة Python؛ Node.js مع إطار Django، Flask؛ Ruby مع إطار Ruby on Rails؛ PHP مع إطار Laravel وتُمكن هذه الأدوات المطورين من تنفيذ منطق تشغيلي معقد، وتحقيق

قابلية توسع عالية، فضلاً عن دعم بيئات الويب الحديثة (Quvvatov, 2024; Yanuarsyah et al., 2024)

- **نظم إدارة قواعد البيانات (DBMS):** يؤثر اختيار نظام إدارة قاعدة البيانات - سواء أكان علائقيًا مثل MySQL, PostgreSQL أم غير علائقي مثل MongoDB تأثيرًا مباشرًا على هياكل تخزين البيانات واستراتيجيات إدارتها. وتُعد النظم العلائقية (Relational DBMSs) خيارًا مناسبًا تمامًا للبيانات المهيكلة، في حين تُسهل حلول NoSQL التعامل مع كميات ضخمة من البيانات غير المهيكلة، وهو أمر يكتسب أهمية خاصة بالنسبة لتطبيقات الويب التي تتطور بشكل ديناميكي (Quvvatov, 2024)

- **واجهات برمجة التطبيقات وحلول التكامل (API and Integration Solutions):** تُستخدم الواجهات القياسية، التي يتم تنفيذها عبر أنماط معمارية مثل REST و Graph QL، على نطاق واسع لتحقيق اتصال فعال بين أنظمة الواجهة الأمامية والخلفية. إذ يوفر نمط REST تنسيقًا مباشرًا وسهل الفهم لتبادل البيانات، في حين يتيح Graph QL استرجاع البيانات المطلوبة فقط؛ مما يعزز الأداء العام (Goh et al., 2023; Sharipova, 2023).

في هذا البحث تم الاعتماد على لغة الاستعلامات المهيكلة (SQL)، ولغة البرمجة (PHP)، وخادم XAMPP. فيما يلي عرض مفصل لهم:

#### - لغة الاستعلامات المهيكلة (SQL):

ذكر (Siregar et al., 2024) أن لغة الاستعلام المهيكلة (SQL) لغة برمجة خاصة مصممة للوصول إلى البيانات وإدارتها في قواعد البيانات العلائقية وتُعتبر المعيار القياسي لإدارة قواعد البيانات، حيث تدعمها جميع خوادم قواعد البيانات تقريبًا. وتتكون SQL عادةً من أوامر بسيطة، تُعرف غالبًا باسم الاستعلامات وتحتوي على تعليمات لمعالجة واسترجاع البيانات من قواعد البيانات المنظمة.

وأشار (Rockoff, 2017) إلى أن SQL هي لغة حاسوبية قياسية للحفاظ على البيانات واستخدامها في قواعد البيانات العلائقية، كما أن لها تاريخ طويل من التطوير من قبل منظمات مختلفة يعود إلى سبعينيات القرن الماضي.

وتبرز أهمية لغة الاستعلامات المهيكلية (SQL) في قدرتها على التعامل مع كميات هائلة من البيانات قد تصل إلى تيرابايت، متجاوزةً بذلك قيود الأدوات التقليدية مثل جداول البيانات؛ مما يمنح المبرمجين والمحللين تحكماً دقيقاً يعزز من كفاءة الأداء وسرعته ودقته. (DeBarros, 2018, p. 15)

كما تنسم لغة (SQL) بمجموعة من الخصائص الجوهرية؛ فإلى جانب توحيد الأوامر الأساسية عبر مختلف أنظمة الإدارة، التي تسهل على المطورين التنقل بين بيئات العمل المختلفة، توفر SQL بنية قوية لضمان سلامة البيانات وتكاملها عبر آليات دقيقة مثل المفاتيح الأساسية (Primary Keys) التي تضمن تقرد كل سجل، والمفاتيح الخارجية (Foreign Keys) التي تفرض علاقات مترابطة ومنطقية بين الجداول. وتتقدم بأدوات متطورة لتبسيط العمليات المعقدة؛ فمفاهيم مثل العروض (Views) تتيح تجريد الاستعلامات المعقدة في صورة جداول افتراضية؛ مما يعزز من تنظيم الكود ويفرض طبقة من الأمان عبر تقييد الوصول للبيانات الحساسة، بينما تعمل الإجراءات المخزنة (Stored Procedures) على تجميع منطق العمل في وحدات قابلة لإعادة الاستخدام؛ مما يرفع من كفاءة النظام وقابليته للصيانة. (Chan, 2018)

كما يُقدم النموذج العلائقي، الذي تُعد لغة SQL أدواته القياسية، إطاراً ناضجاً وموثوقاً لإدارة البيانات، ويتميز بقدرته على ضمان اتساق وسلامة البيانات من خلال دعمه لخصائص المعاملات (ACID). وتبرز كفاءته بشكل خاص في البيئات التي تتطلب استعلامات معقدة وتقارير دقيقة، حيث يتفوق على بدائله في تنفيذ عمليات الربط (Joins) والاستعلامات المعقدة. وعلى الرغم من أن قابلية التوسع الرأسي قد تمثل تحدياً من حيث التكلفة مقارنةً بالحلول الأخرى، إلا أن هذا النموذج يظل الخيار الأفضل عند الحاجة إلى ضمان اتساق قوي للبيانات، مما يجعله الحل الأمثل للعديد من التطبيقات المؤسسية الحيوية (Shareef et al., 2022).

كما تتجاوز SQL دورها كمجرد أداة للاستعلام لتصبح مكوناً استراتيجياً في تمكين اتخاذ القرارات المعتمدة على البيانات؛ فهي تسمح للمحللين بربط مصادر بيانات متعددة، وتجميع وتصفية المجموعات الضخمة بكفاءة. كما أنها تشكل الطبقة الأساسية التي تعتمد عليها أدوات ذكاء الأعمال المتقدمة ومنصات التحليل السحابية، حيث تسهل عمليات استخراج البيانات وتحويلها ونقلها، وتدعم إنشاء لوحات المراقبة اللحظية، مما يحول البيانات الأولية إلى رؤى استراتيجية قابلة للتنفيذ. (Al Maruf et al., 2022)

أما عن لغة PHP تُعرّف بأنها لغة برمجة نصية شائعة وقوية تعمل من جانب الخادم (server-side)، وتُعتبر أداة فعالة لتطوير تطبيقات ومواقع الويب الحديثة، الديناميكية، والمعتمدة على البيانات. وهي



من أكثر اللغات شيوعًا في مجال تطوير الويب وتُستخدم بشكل أساسي في بناء وتنفيذ الواجهات الخلفية (backend) للمواقع. (Novaliendry et al., 2021)

وتُعد لغة PHP أداة برمجية قوية وراسخة في مجال تطوير مشاريع الويب الديناميكية، وتستمد شعبيتها الواسعة من مجموعة من الخصائص الجوهرية التي تشمل البساطة، والكفاءة الاقتصادية، وقابلية التوسع، والتوافقية العالية. وتتجلى مرونتها في طبيعتها الديناميكية التي تسمح بإجراء تعديلات في أي مرحلة من مراحل التطوير دون إهدار للوقت، بالإضافة إلى سهولة تكاملها مع لغة HTML. وتُعزز هذه القوة من خلال توافقها مع مختلف أنظمة التشغيل والخوادم، ودعمها الأصلي لقواعد البيانات الشائعة مثل: MySQL؛ مما يجعلها مدعومة من غالبية مزودي خدمات الاستضافة، وغالبًا دون تكلفة إضافية. وقد أثبتت اللغة فعاليتها وقدرتها على بناء مشاريع معقدة وواسعة النطاق، حيث استُخدمت في تطوير مواقع عالمية كبرى مثل: فيسبوك وويكيبيديا (Sotnik et al., 2023)، وذكر (Amini et al., 2021) أن هذا ما يجعلها الخيار الأفضل والأنسب الذي بُنيت عليه أطر عمل متقدمة تهدف إلى تحسين جودة البرمجيات.

وتُعد لغة PHP من أكثر لغات البرمجة استخدامًا في تطوير تطبيقات الويب، ويرجع ذلك إلى تكاملها الفعّال مع نمط معمارية MVC الذي يوفر تنظيمًا دقيقًا لمكونات التطبيق. حيث تُسهّم هذه البنية في فصل واضح بين منطق البيانات والعرض والتفاعل؛ مما يعزز من قابلية الصيانة ويُسهّل عمليات التوسعة والتعديل لاحقًا. ومن بين أبرز ما يُميز PHP أيضًا هو دعمها لإعادة استخدام الشفرة البرمجية وتنظيمها بطريقة تجعلها مرنة للتطوير المستمر، وهو ما ينعكس إيجابًا على سرعة الإنجاز واستقرار الأداء. كما أنها تتيح العمل التعاوني بين فرق التطوير، بحيث يمكن للمصمم والمطور ومهندس قاعدة البيانات العمل بالتوازي دون تعارض، مما يقلل الاعتماد المتبادل ويزيد الإنتاجية. إلى جانب ذلك، أظهرت الدراسات أن استخدام PHP ضمن إطار عمل منظم يُسهّم في تعزيز الأمان، من خلال تقليل فرص حدوث ثغرات مثل الهجمات عبر المتصفح، بفضل التقسيم المنهجي للمهام والوظائف. هذه الخصائص تجعل من PHP خيارًا مناسبًا لبناء تطبيقات ويب قوية، آمنة، وقابلة للتطوير على المدى الطويل (Khan & Khanam, 2023).

**تتسم لغة PHP بمزايا جمّة، من أبرزها:** الاعتمادية والموثوقية العالية، حيث أثبتت كفاءتها في تطوير مواقع ديناميكية وتفاعلية، إضافةً إلى كونها منخفضة التكلفة مقارنة بلغات أخرى، مما يجعلها مناسبة للشركات



الصغيرة والمتوسطة. كما تتميز PHP بسهولة التعلم والاستخدام، الأمر الذي يُمكن المطورين من بناء مواقع إلكترونية بسرعة وكفاءة. تدعم اللغة عددًا كبيرًا من قواعد البيانات وتُعد مرنة وقابلة للتكامل مع تقنيات أخرى، مما يمنح المطورين حرية أكبر في البناء والتخصيص. وقد أظهرت تجربة MahamGostar أن PHP توفر بيئة ملائمة لفرق التطوير الصغيرة ذات الموارد المحدودة، دون فقدان الأداء أو الأمان (Amini et al., 2021).

#### • أهمية تنمية مهارات برمجة تطبيقات الويب التعليمية :

أشار (الطويلة، 2025) إلى أهمية تنمية مهارات برمجة تطبيقات الويب التعليمية في النقاط التالية:

- يُعد تعلم البرمجة الوسيلة الوحيدة لإنتاج تطبيقات تعليمية عالية الجودة والكفاءة، مقارنةً بالأدوات الجاهزة محدودة القدرة.
- تعتبر برمجة تطبيقات الويب التعليمية جزءًا أصيلاً ومهماً من برنامج إعداد طلاب الحاسب الآلي، ومهمة أساسية من مهامهم الوظيفية المستقبلية.
- العمل على تنمية مهارات التحليل والاستنتاج والربط بين البيانات من خلال الأكواد والتعليمات البرمجية، وتنمية القدرة على وضع بدائل لحل المشكلات واختيار أفضلها.
- تنمية التعلم الذاتي حيث تساعد البرمجة على تنمية مهارات التعلم الذاتي لدى المتعلمين، بالإضافة إلى إكسابهم المعارف والمهارات المرتبطة بلغات البرمجة.
- تعزيز الجوانب الشخصية والإبداعية مما يزيد من ثقة المتعلم بنفسه، وتشجعه على الاستقلالية، كما تعزز مهارات تفكيره الإبداعي وتنشط قدراته العقلية.
- التكيف التكنولوجي الذي يتيح للمتعلم فرصة فهم التكنولوجيا وكيفية التعامل معها؛ مما يكسبه القدرة على التكيف مع المتغيرات التكنولوجية المستقبلية.
- حل المشكلات التي تعمل على تحسين التفكير الإبداعي من خلال تدريب المتعلمين على إيجاد حلول مبتكرة للمشكلات المختلفة.
- التكامل المعرفي الذي تعزز فهم واستيعاب المواد الدراسية والمقررات الأخرى؛ وذلك من خلال تحسين القدرة على الاستيعاب والتفكير بطريقة أكثر منهجية وتحليلية.